

Comp 3016 CW1 Proposal

Game Title

The Scarygon Trail

Genre

The story of the game is generative, combining comic horror and survival resources management storytelling, inspired by The Oregon Trail.

Synopsis

Your wagon company is going toward the west in the early 1800s. Nothing in that sentence has prepared you for everything that will happen to you after that journey begins. Each stop along the road tells us a brief narrative incident – with the same tone, whether it is a broken wheel or your son's attempt to defy Christianity and try to place a hex on you. The dread is genuine, the stakes are sharp, but the game never looks at you in the eye, it is up to your son to do that.

Core Gameplay Loop

1. **Travel** to the next trail stop (SDL3-rendered wagon moving along a map).
2. **View status** — top panel shows party members, inventory and wagon condition at a glance.
3. **Resolve an event** — bottom panel presents narrative text and 2-4 button choices.
4. **Apply consequences** — resources, sanity, or party members change based on the choices made.
5. Repeat until the party reaches the end of the trail or is wiped out.

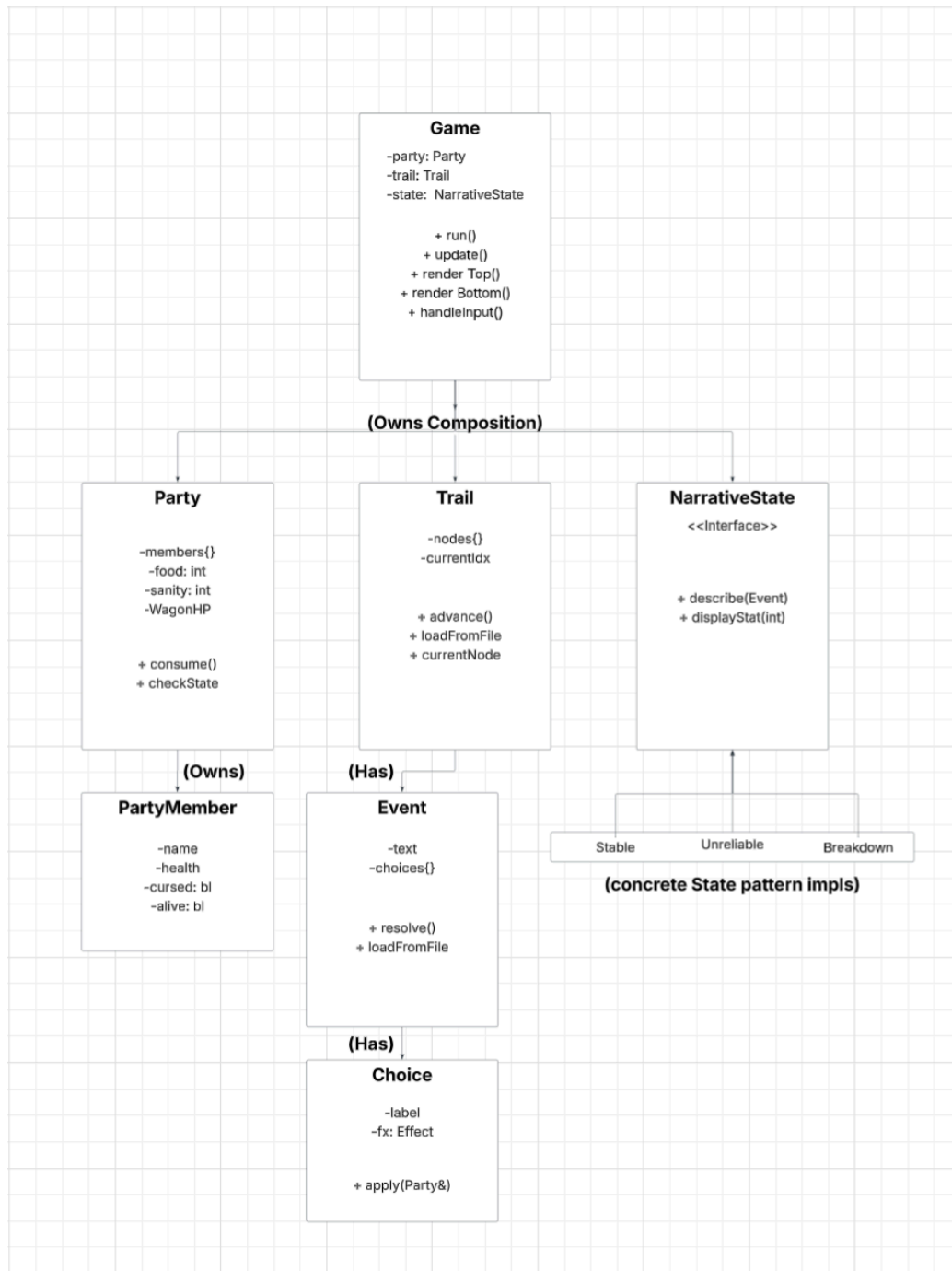
Intended Mechanics

- **Resource management:** food, health, sanity, wagon condition — Oregon-Trail-style erosion over distance and events.
- **Escalating absurdist event table:** events start mundane (low supplies, sick ox) and escalate toward the surreal/horrific (a cursed heirloom, a trail cult, things that shouldn't be in the woods)
- **Sanity-driven unreliable narration:** at low sanity, event text and even displayed resource numbers become unreliable — the game itself starts lying, played for dark comedy rather than pure dread.
- **Branching choice consequences:** each event's button choices have different risk/reward outcomes (e.g., repair the wagon wheel yourself vs. trust the stranger who offers to help).
- **Party member individuality:** named party members can each be affected independently (sick, cursed, lost, dead), not just a shared health pool.

Intended Technical Features

- **C++ with SDL3** for rendering and input.
- **Split UI layout:** top viewport for 2D visuals (wagon, party sprites, inventory icons); bottom viewport for scrolling text and button-based choice input.
- **File-based event/content loading:** event text, choices and outcomes loaded from external data files at runtime (keeps writing/editing content separate from code, and satisfies the file-loading expectation).
- **OOP architecture:** Party, PartyMember, TrailNode, Event, Choice classes with clear responsibilities.
- **State pattern** for sanity-driven narrative states (stable / unreliable / breakdown).
- **Observer pattern** for resource thresholds triggering event-table shifts (e.g., low food unlocking desperate-choice events).
- Defensive input handling throughout, so no malformed file or input can crash the game.

UML Sketch



Architecture Overview

- The main loop is managed by the Game class which coordinates between the rendering processes (the SDL3 visual panel on the top and the text and choice panel below) and logic updates.
- The Trail class maintains a list of TrailNode objects, with each node referring to an Event loaded from the file.
- An Event object contains texts and a list of Choice objects, where each option has its own effects influencing the resources and sanity of a character depending on the choice made.

- The Party object itself is made up of several PartyMember objects (in terms of composition, the Party creates and destroys the members) keeping the track of shared resources and states of the members.
- The NarrativeState class uses State pattern design to define how reliable the event text and the quantities of resources are displayed and switched depending on the sanity of the Party.

External Libraries

- **SDL3** (required — rendering, window, input)
- No further libraries currently planned; will request approval for additions (e.g. irrKlang for sound) if pursued later.

Intended Use of AI

AI (ChatGPT) will be used for:

- Drafting and refining absurdist/comedic event text and branching choice outcomes
- Idea generation for the escalating event table
- Generating segments of code to save time
- Debugging and review support
- Documentation/report structuring assistance

In accordance with the requirements stated in the brief, all applications shall be recorded in the AI Development Log with necessary tools, clue, classifications of use, and means of verification.

Development Roadmap

Week	Milestone
1	Plan 30% and proposal finalizing.
2	Proposal submitted/split-panel skeleton and SDL3 window.
3	Trail/Party/Event classes made/ file-based event loading/first git commits.
4	Core travel loop works/basic rendering panel for top/bottom.
5	Choice system with consequences/sanity mechanic/unreliable narration.
6	(Reading week) Full event table and playtesting round 1. (bug fixing)
7	Polising/balancing/log completion for AI/report/video. (Submit)

